DESIGN PATTERNS & FRAMEWORKS

LECTURE 05

ADAPTER DESIGN PATTERN

Making Incompatible Objects Work Together

STRUCTURAL DESIGN PATTERN



Previously Covered

SINGLETON

FACTORY

FACADE

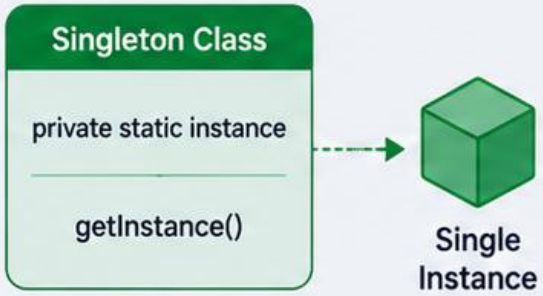
"Don't change existing code. Adapt it."

Previous Lecture Recap

Creational Patterns

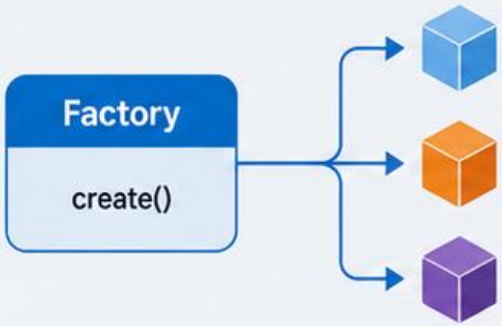
Singleton

One object



Factory

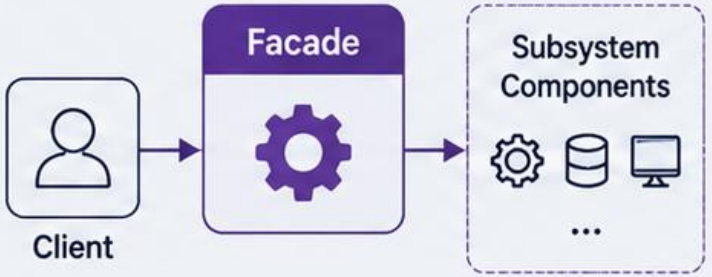
Create objects



Structural Pattern

Facade

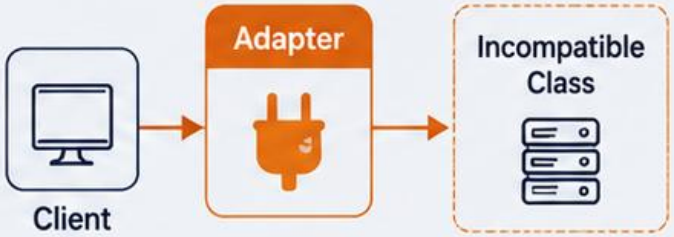
Simplify a complex system



Today's Pattern

Adapter

Make incompatible classes communicate



Different Problems. Different Solutions. **Better Design.**

Learning Outcomes

After this lecture students should be able to

01 Explain Adapter Pattern

02 Identify Adapter situations

03 Design Target, Adapter and Adaptee

04 Implement Adapter in Java

05 Compare Adapter vs Facade

> *Learn it. Design it. Code it. Compare it.*



Adapter makes incompatible classes work together **without changing their code.**

Real Life Examples

1 USB-C → HDMI



USB-C → HDMI

2 USB-C → VGA



USB-C → VGA

3 Travel Plug



4 Memory Card Reader



5 Bluetooth Receiver



Ask students

What is common in all these examples?

Answer

"They connect **incompatible interfaces.**"

Different devices. Different interfaces. Same solution – **Adapter Pattern!**

The Problem

1. Imagine you already have



Printer

printDocument()

2. Now your company purchases a new printer



printPDF()

3. Old system expects



printDocument()

4. New printer provides



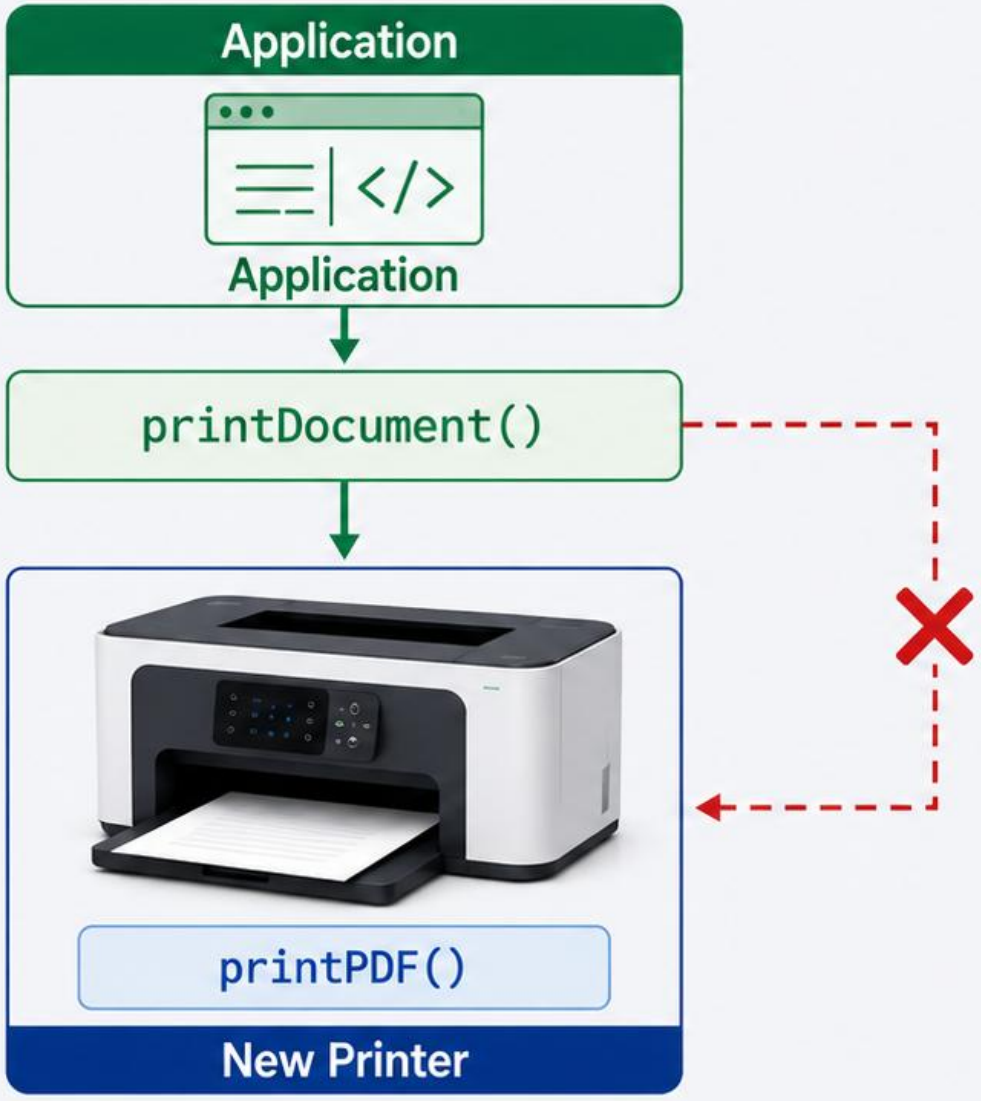
printPDF()

Should we rewrite the whole application? ✗ **No.** → **Use Adapter.**

Key Idea: Adapter lets incompatible interfaces work together without changing the existing system.

Old System (printDocument()) ⇢ Adapter ⇢ New Printer (printPDF())

Without Adapter



Compilation Error

Different interfaces

```
error: cannot find symbol
 method printDocument()
 location: class NewPrinter
 required: printPDF()
 found:    printDocument()
```

Adapter Solution

Application calls `printDocument()`

1. Application

</>

Old system expects `printDocument()`

2. Printer Interface

`printDocument()`
(Expected by old system)

Client works with this interface

Adapter converts `printDocument()` to `printPDF()`

3. Printer Adapter

Implements `printDocument()` and internally calls `printPDF()`

Adapter bridges the gap

Modern printer understands `printPDF()`

4. Modern Printer (Adaptee)



`printPDF()`
(Provided by new printer)

How it works

1. Application calls `printDocument()`.
2. Adapter implements `printDocument()`.
3. Adapter converts the request and calls `printPDF()` on the modern printer.
4. Modern printer performs the task.

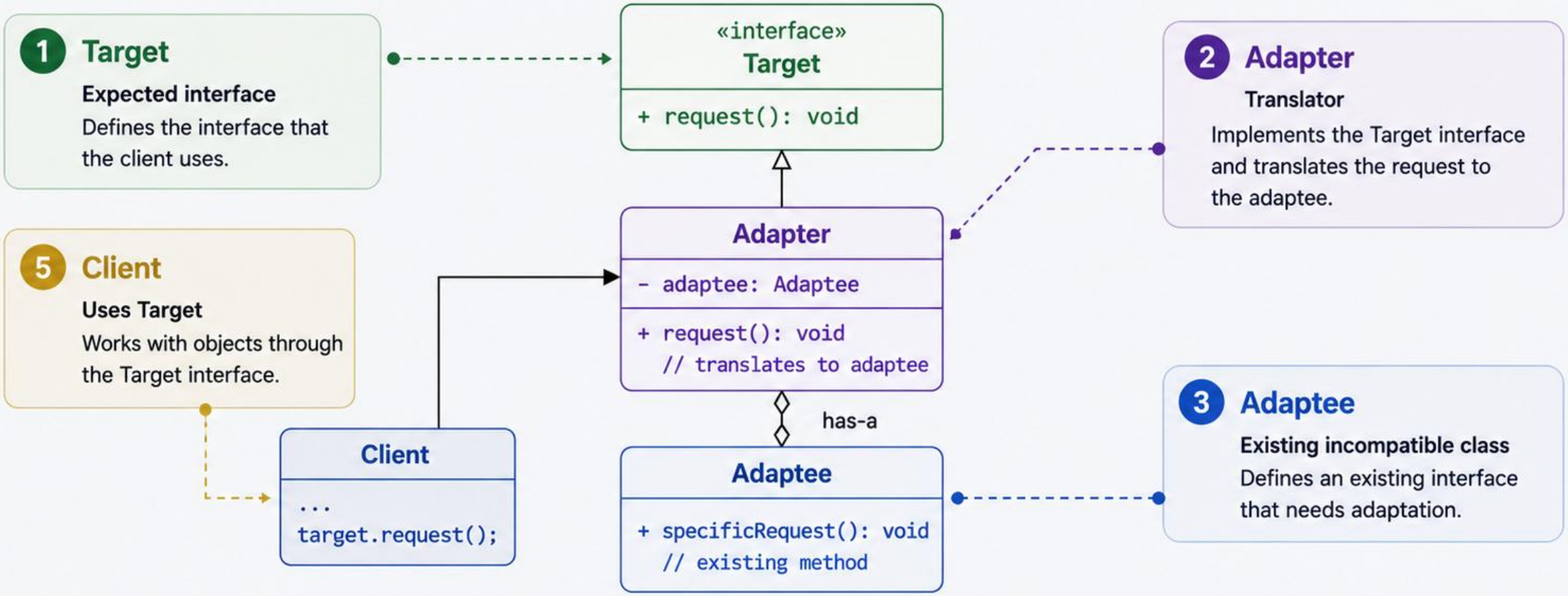
Benefits

- No changes to existing application
- Works with new printer
- Open for extension, closed for modification

Result: Incompatible interfaces work together using Adapter.

Application → Printer Interface → Adapter → Modern Printer (printPDF())

Adapter Participants



Summary

Adapter acts as a bridge between the Client and the Adaptee.
It implements the Target interface and translates requests into a form understandable by the Adaptee.

1 Target	Expected interface
2 Adapter	Translator
3 Adaptee	Existing incompatible class
5 Client	Uses Target

Java Example Overview

We will build: `Printer` → `PrinterAdapter` → `ModernPrinter`

Participants

Printer (Target)
Expected interface
Defines what the client uses.

PrinterAdapter (Adapter)
Translator
Implements `Printer` and adapts `ModernPrinter`.

ModernPrinter (Adaptee)
Existing incompatible class
Provides a different interface.

Client
Uses Printer
Works with the target interface.

1 Printer (Target)

```
public interface Printer {
    void printDocument(String
      document);
}
```

Example Usage (Client)

```
Printer printer = new
    PrinterAdapter(new
      ModernPrinter());

printer.printDocument("Report.pdf");
// Works without changing client code
```

2 PrinterAdapter (Adapter)

```
public class PrinterAdapter
    implements Printer {
    private ModernPrinter
      modernPrinter;

    public PrinterAdapter(
      ModernPrinter modernPrinter) {
        this.modernPrinter =
          modernPrinter;
    }

    @Override
    public void printDocument(String
      document) {
        // Adapt the call
        modernPrinter.printPDF(
          document);
    }
}
```

3 ModernPrinter (Adaptee)

```
public class ModernPrinter {
    public void printPDF(String
      pdfContent) {
        System.out.println(
          "Printing PDF: "
          + pdfContent);
    }
}
```

Flow

1. Client calls `printDocument()` on `Printer` interface.
2. **PrinterAdapter** receives the call and translates it.
3. Adapter calls `ModernPrinter.printPDF()` internally.
4. **ModernPrinter** performs the actual PDF printing.

1 Step 1 – Target Interface

The **Client** works only with this interface.

Target Interface

```
public interface Printer {

    void print(String text);

}
```

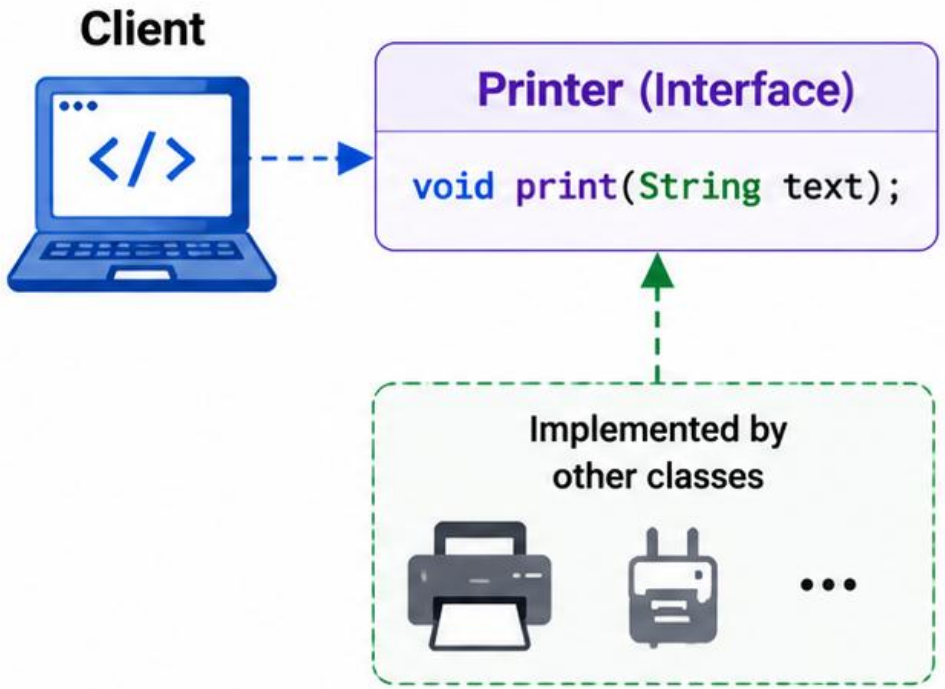
- Defines what the client expects.
- This is the contract (client depends on this).
- Any class that acts as a `Printer` must implement this.

Example Usage (Client)

```
Printer printer = getPrinter();
printer.print("Hello World");
```

- The client doesn't know or care which actual class will do the printing.
- It only knows `Printer` interface.

Who uses it?



Key Takeaway

The **Target** interface acts as a contract between **Client** and the rest of the system.

This is the foundation of the **Adapter Pattern.**

2 Step 2 – Adaptee

Adaptee – ModernPrinter

```
public class ModernPrinter {

    public void printPDF(String file) {

        System.out.println("Printing PDF : " + file);
    }

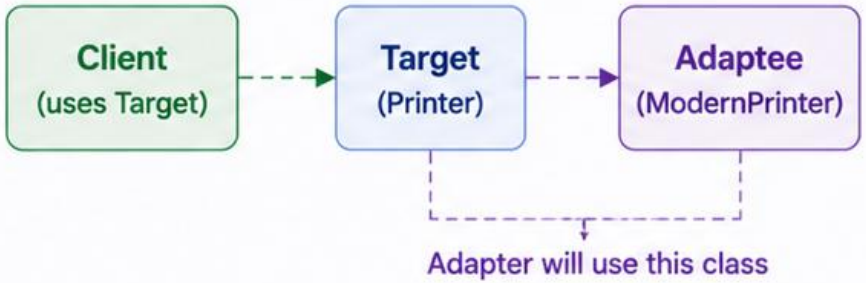
}
```

This is an existing class we cannot change.
It provides its own interface `printPDF(String file)`.

Role: Adaptee

The existing class with an incompatible interface that the adapter needs to work with.

Where it fits



Notice

No `print()` method.

It has only `printPDF(String file)`.

3 Step 3 – Adapter

Adapter – PrinterAdapter

```
public class PrinterAdapter implements Printer {

    private ModernPrinter printer;

    public PrinterAdapter(ModernPrinter printer) {
        this.printer = printer;
    }

    @Override
    public void print(String text) {
        // Translate print(text) to printPDF(text)
        printer.printPDF(text);
    }
}
```

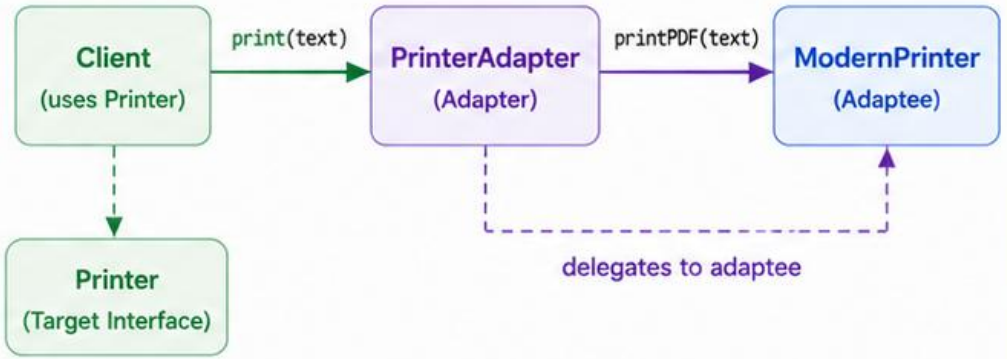
This is the heart of Adapter.

It implements the Target interface (Printer) and translates the request into a call that the Adaptee (ModernPrinter) understands.

What it does

- Implements the Target interface (Printer).
- Holds a reference to the Adaptee (ModernPrinter).
- Converts the request from the client into a format the Adaptee can understand.

How it works



Key Point

Now any client using Printer can work with ModernPrinter through PrinterAdapter.

4 Step 4 – Client

Client

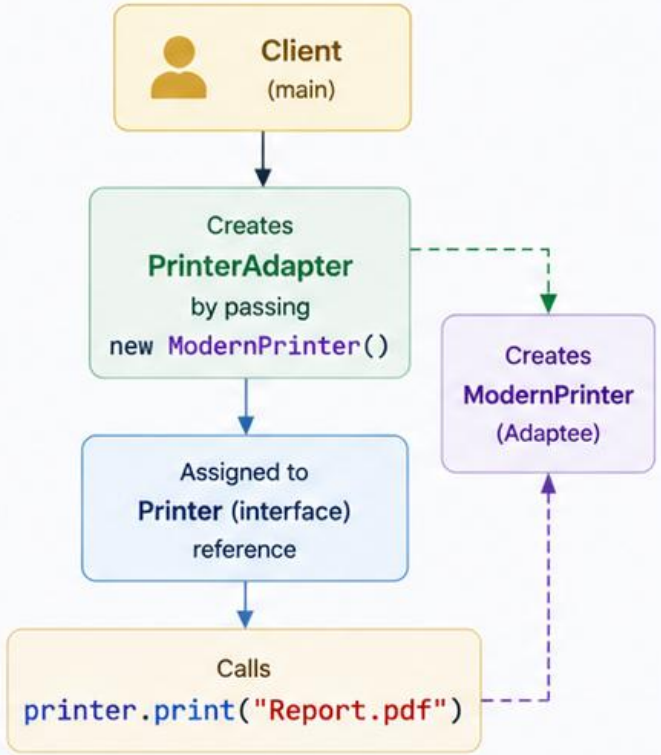
```
public class Client {

   public static void main(String[] args) {

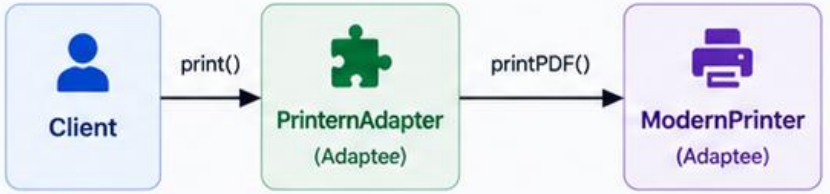
      Printer printer =
          new PrinterAdapter(
              new ModernPrinter()
          );

      printer.print("Report.pdf");
   }
}
```

What happens in main()



Call Flow



1. Client calls `print("Report.pdf")` on `Printer` interface.
2. `PrinterAdapter` receives the call and translates it.
3. Adapter calls `ModernPrinter.printPDF("Report.pdf")` internally.
4. `ModernPrinter` performs the actual PDF printing.

Output

```
Printing PDF : Report.pdf
```

Key Takeaway

The client works only with the `Printer` interface.

Thanks to `PrinterAdapter`, it can use `ModernPrinter` without knowing its different interface.

What Happened?

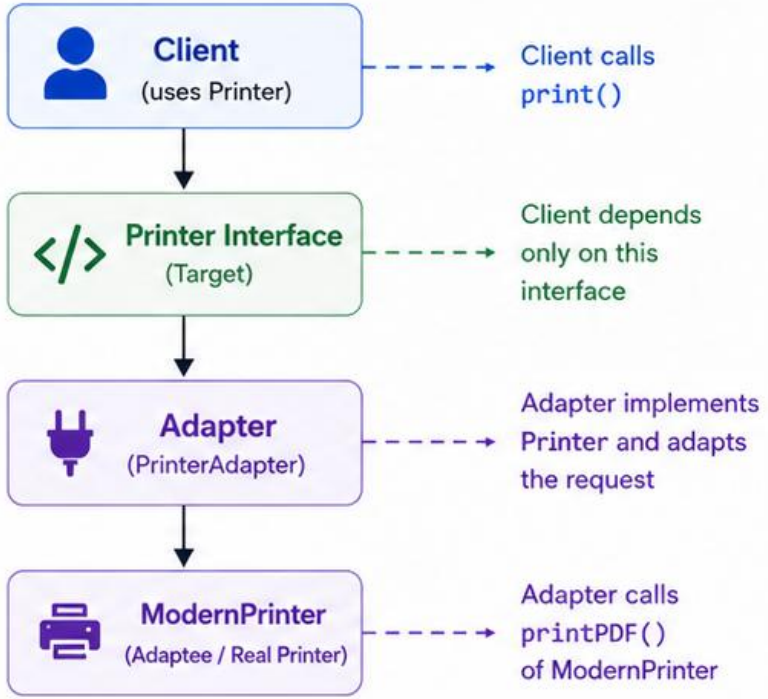
Client
The client wants to print a document.

Printer Interface
The client depends only on the **Printer** interface.

Adapter
The adapter implements the **Printer** interface and adapts the request.

ModernPrinter
The adapter forwards the request to the real printer (**ModernPrinter**).

How it Works



The client communicates through the interface.
The real printer is hidden behind the adapter.

What Really Happened?

1. Client created a `Printer` reference that points to a `PrinterAdapter` object.
2. When `print()` is called, the call goes to `PrinterAdapter`.
3. `PrinterAdapter` translates the call and delegates it to `ModernPrinter`.
4. `ModernPrinter.printPDF()` is executed, and the document is printed.

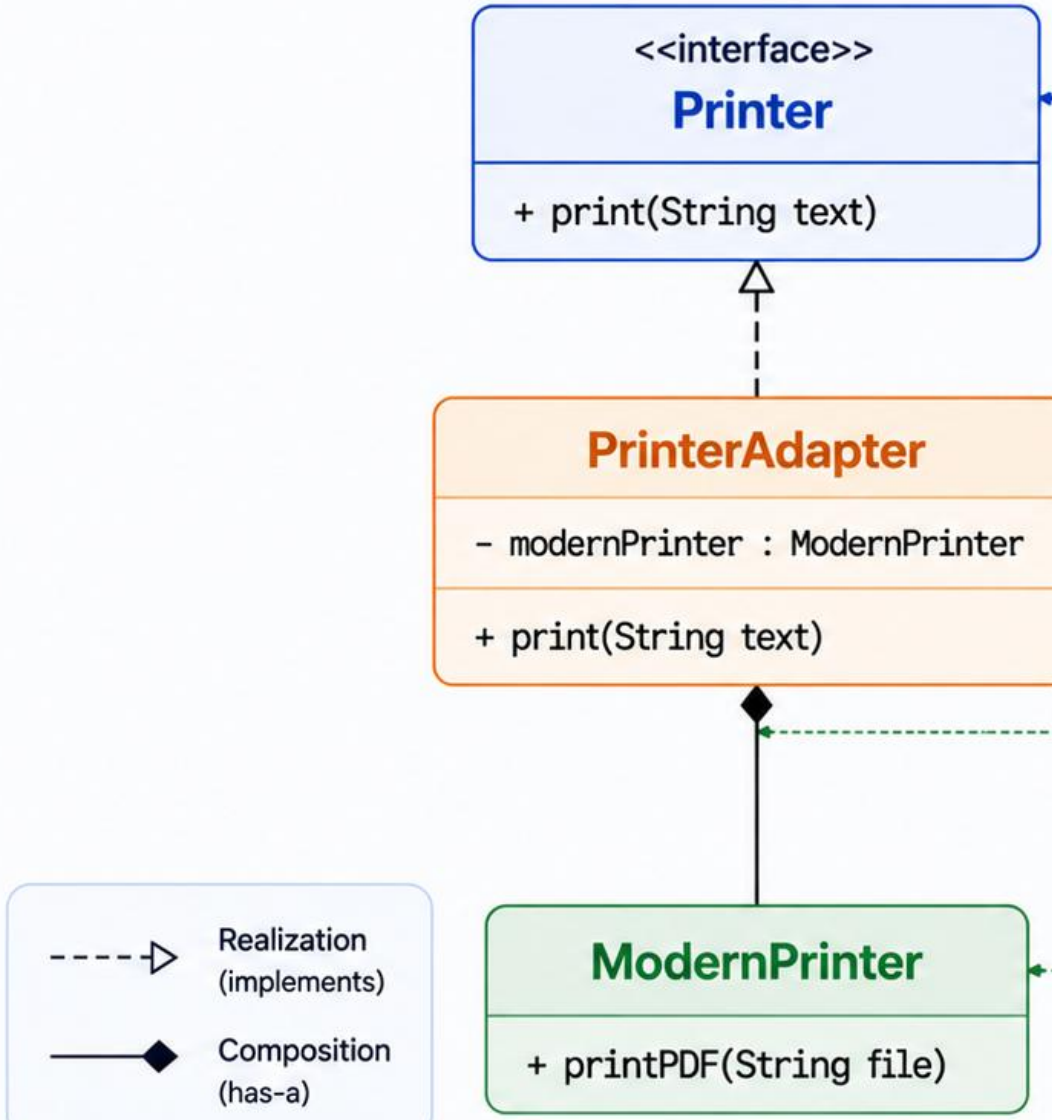
Key Point

> **The client never knows the real printer.**



This is the essence of the Adapter Pattern – **Interface stays the same, implementation is made compatible.**

UML Diagram – Adapter Design Pattern



Target
Defines the interface the client uses.

Adapter
Implements the Target interface and adapts the request.

Has-a relationship
Adapter holds an instance of the Adaptee (ModernPrinter).

Adaptee
Existing class that has an incompatible interface.

Legend

Target
The interface the client expects.

Adapter
Adapts the request to the Adaptee and implements the Target interface.

Adaptee
The existing class with an incompatible interface.

Key Point

The Adapter lets classes work together that couldn't otherwise because of incompatible interfaces.

Adapter Design Pattern – Payment Gateway Example

Making New Gateway work with Old System

1 Old System (Target)

The existing system expects a method `pay(amount)`.

```
public interface PaymentGateway {
    void pay(double amount);
}
```

2 New Gateway (Adaptee)

The new payment gateway has a different method `makePayment(value)`.

```
public class NewGateway {
    public void makePayment(double value) {
        System.out.println("Payment of $" + value + " processed by New Gateway");
    }
}
```

3 Adapter (PaymentAdapter)

The adapter implements the old interface and internally uses the new gateway.

```
public class PaymentAdapter implements PaymentGateway {
    private NewGateway newGateway;

    public PaymentAdapter(NewGateway newGateway) {
        this.newGateway = newGateway;
    }

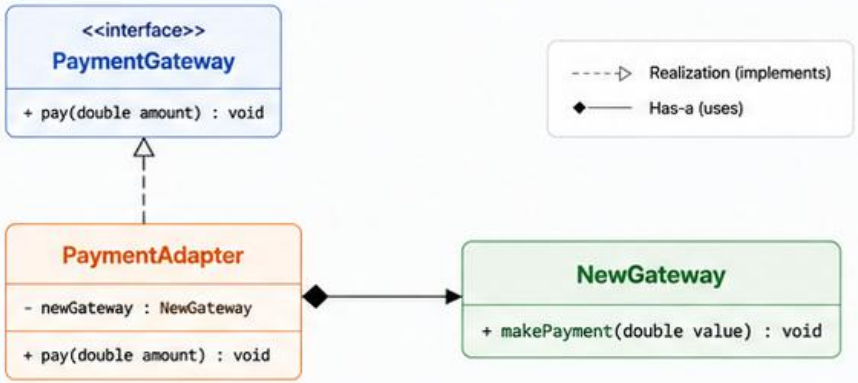
    @Override
    public void pay(double amount) {
        // Adapting the call to new gateway method
        newGateway.makePayment(amount);
    }
}
```

How It Works



1. Old System calls `pay(amount)` on `PaymentGateway` interface.
2. `PaymentAdapter` receives the call and translates it.
3. `PaymentAdapter` calls `makePayment(value)` on `NewGateway`.
4. `NewGateway` processes the payment.

UML Class Diagram



4 Client Code

```
public class Client {

    public static void main(String[] args) {
        PaymentGateway gateway =
            new PaymentAdapter(new NewGateway());

        gateway.pay(150.75);
    }
}
```

Output

```
Payment of $150.75
processed by New Gateway
```

Key Takeaway

The old system works with the new payment gateway through the adapter without changing any existing code.

Same interface, different implementation!

Adapter Pattern lets you use a new class (NewGateway) where an old interface (PaymentGateway) is expected.

Payment Adapter Code

Adapter Design Pattern in Java – Making New Gateway work with Old System

1 Target Interface

```
interface Payment {
      void pay(double amount);
}
```

The old system expects this interface.

2 Adaptee (New Payment Gateway)

```
class Stripe {
    public void makePayment(double value){
        System.out.println(value);  // New gateway method
    }
}
```

The new gateway has a different method name.

3 Adapter

```
class StripeAdapter implements Payment{
    private Stripe stripe;

    public StripeAdapter(){
        stripe = new Stripe();        // Create adaptee
    }

    public void pay(double amount){
        stripe.makePayment(amount);   // Delegate to adaptee
    }
}
```

Implements the old interface and adapts the call to the new gateway.

How It Works



1. Client calls `pay(amount)` on `Payment` interface.
2. `StripeAdapter` receives the call and delegates it to `Stripe`.
3. `Stripe` executes `makePayment(value)`.

Key Points

- `Payment` is the old interface expected by the client.
- `Stripe` is the new payment gateway with a different method (`makePayment`).
- `StripeAdapter` acts as a bridge between the client and the new gateway.
- The client code remains unchanged.

Example Output

```
// Client Code
Payment payment = new StripeAdapter();
payment.pay(250.75);

250.75
```

The client uses `Payment` interface, but the call is handled by the new `Stripe` gateway through the adapter.

Java Library Examples – Adapter Pattern

The Java standard library uses the Adapter pattern in many places!

Example 1

java.io.InputStreamReader

Adapts **InputStream** to **Reader**




```
InputStream in = new FileInputStream("data.txt");
Reader reader = new InputStreamReader(in);
```

Example 2

Arrays.asList()

Converts **Array** to **List**




```
String[] names = {"Alice", "Bob", "Charlie"};
List<String> list = Arrays.asList(names);
```

Example 3

Collections.enumeration()

Converts **Collection** to **Enumeration**

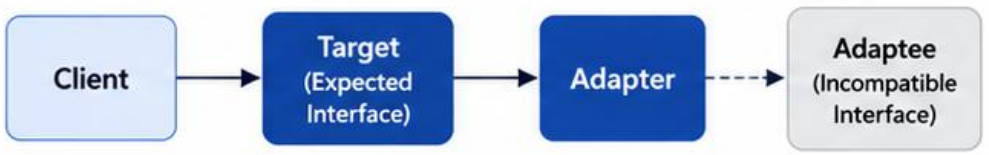



```
List<String> list = Arrays.asList("A", "B", "C");
Enumeration<String> e =
   Collections.enumeration(list);
```

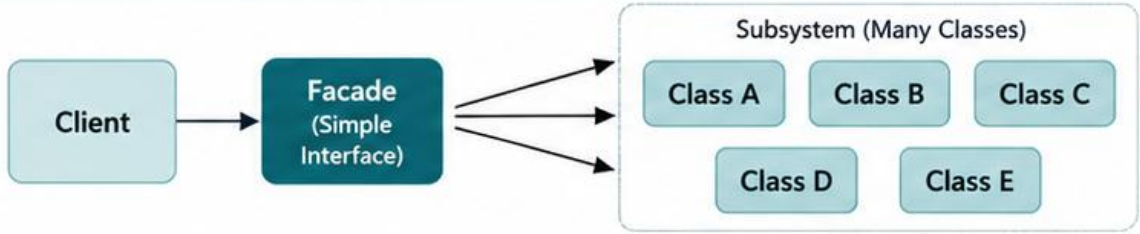
These classes act as **adapters**, allowing incompatible interfaces to work together without changing existing code.

Adapter vs Facade

ADAPTER		FACADE
Makes incompatible interfaces compatible	Purpose	Simplifies a subsystem
One object	Scope	Many objects
Works with existing systems and classes	Focus	Provides a new simplified interface to a subsystem
Translator	Role	Simplifier



Adapter acts as a bridge between the client and the incompatible system.



Facade provides a unified and simplified interface to a complex subsystem.

In Short: Adapter makes two incompatible interfaces work together. | Facade makes a complex system easy to use.

Adapter Design Pattern – Pros & Cons

Weighing the Benefits and Trade-offs

Advantages

Reuse legacy systems

Allows existing classes with incompatible interfaces to be reused without modification.

Loose coupling

Client code depends on the target interface, not the concrete adaptee.

Open/Closed Principle

New adapters can be introduced without changing existing client code.

Easy integration

Helps integrate new components or third-party libraries smoothly.

Easy testing

Adapters enable mocking or stubbing the adaptee, making unit testing easier.

Disadvantages

More classes

Introduces additional adapter classes, increasing the number of classes in the system.

Extra layer

Adds an extra abstraction layer between the client and the actual functionality.

Can become confusing if overused

Too many adapters can make the design harder to understand and maintain.

Small performance overhead

The extra layer of indirection may introduce a slight performance overhead.

Key Takeaway: The Adapter pattern is powerful for making incompatible components work together, but should be used judiciously to avoid unnecessary complexity.



Design Pattern Comparison

Quick reference guide to common structural patterns

Pattern	Purpose
Singleton	**One object** Ensures a class has only one instance and provides a global point of access.
Factory	**Create object** Provides an interface for creating objects without specifying the exact class.
Facade	**Simplify subsystem** Provides a unified, simplified interface to a set of interfaces in a subsystem.
Adapter	**Translate interface** Allows incompatible interfaces to work together by acting as a translator.

Remember: Each pattern solves a different problem.

Choose the right tool for the right job!



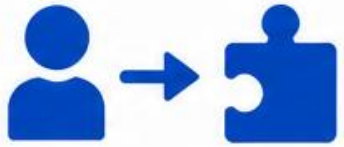
ADAPTER PATTERN QUIZ

Think. Reflect. Understand.

Reflective Questions with Answers

1 Question 1

Does Adapter change the client?



Answer:

No. Adapter allows the client to work with the target interface without any changes. The client remains unaware of the adaptee.

2 Question 2

Who knows the Adaptee?



Answer:

Only the **Adapter** knows the **Adaptee**. It interacts with both the target interface and the adaptee.

3 Question 3

Which pattern simplifies multiple subsystems?



Answer:

Facade Pattern. It provides a single simple interface to a set of interfaces in a subsystem, making the subsystem easier to use.

4 Question 4

Which pattern translates one interface into another?



Answer:

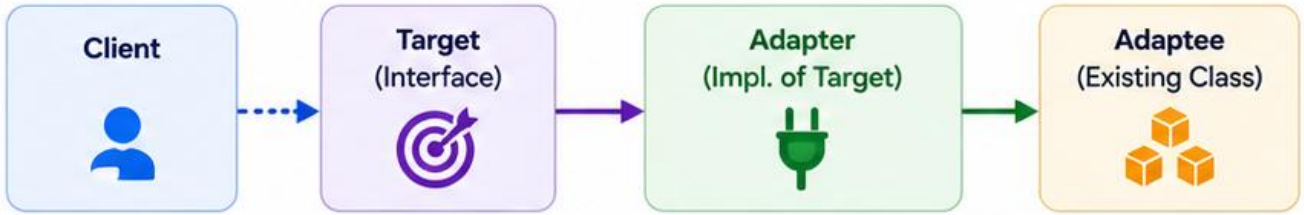
Adapter Pattern. It acts as a translator that converts the interface of a class into another interface that the client expects.

TODAY WE LEARNED

Key takeaways from today's session

Adapter Pattern	Converts the interface of a class into another interface that clients expect.
Target	The interface expected by the client.
Adapter	Implements the Target interface and delegates to the Adaptee.
Adaptee	The existing class with an incompatible interface.
Java Implementation	We built the pattern using Java with interfaces and classes.
UML	We visualized the structure and relationships using UML diagrams.
Adapter vs Facade	Adapter translates interfaces; Facade simplifies complex subsystems.
Real-world applications	Used widely in IO streams, collections, APIs, payment gateways and more.

ADAPTER PATTERN STRUCTURE



Client → Target → Adapter → Adaptee

The client talks to the Target, the Adapter makes it work with the Adaptee.

ADAPTER vs FACADE

ADAPTER	FACADE
Makes incompatible interfaces compatible	Simplifies a subsystem by providing a unified interface
Works with one Adaptee	Works with many subsystems
Focus on interface translation	Focus on interface simplification
Acts as a translator	Acts as a simplifier

REAL-WORLD APPLICATIONS

- **Java I/O Streams** (InputStreamReader, OutputStreamWriter)
- **Collections API** (Arrays.asList, Collections.enumeration)
- **Payment Gateways** (PaymentAdapter)
- **Third-party Libraries & APIs**
- **Legacy Systems Integration**

... and many more!

Key Takeaway:

Adapter Pattern helps us achieve **flexibility and reusability** by making incompatible interfaces work together without changing existing code.

Great work today!

Keep Practicing. Keep Building.